# Plan Técnico 07: Calibración Interactiva y Fusión Dual sin Duplicados

> **ESTADO DEL PROYECTO: IMPLEMENTADO Y OPERATIVO EN HARDWARE DUAL REAL**
> *Fecha de Implementación:* Octubre 2026 | *Hardware Activo:* 2x Intel RealSense D455 ( `207322250777` Master, `203522251709` Slave) | *Puerto Web:* `http://localhost:5002`

## 1. Diagnóstico del Problema: Duplicación Geométrica (Efecto Fantasma)

Al visualizar o capturar con dos cámaras Intel RealSense D455 simultáneas, la escena muestra a la persona o el objeto duplicado (dos cabezas, dos torsos, cuerpos desplazados).

**Causa Raíz Matemática:**

Cada cámara RealSense opera en su propio sistema de coordenadas local tridimensional: * **Cámara 1:** Origen $(0, 0, 0)$ en su sensor de profundidad. * **Cámara 2:** Origen $(0, 0, 0)$ en su propio sensor de profundidad, ubicado físicamente a otra distancia y con otra orientación espacial.

Para co-renderizar ambas nubes en un único espacio unificado, se aplica la transformación euclidiana rígida homogénea $4 \times 4$: $$P_1 = T_{2 \to 1} \cdot P_2 = \begin{bmatrix} R_{3 \times 3} & \mathbf{t}*{3 \times 1} \ *mathbf{0}* \cdot P_2$$} & 1 \end{bmatrix

Donde: * $\mathbf{t} = [t_x, t_y, t_z]^T$ es el vector de traslación física entre sensores (en metros). * $R = R_z(\psi) R_y(\theta) R_x(\phi)$ es la matriz de rotación ortonormal (Yaw, Pitch, Roll).

**Por qué ocurre el error actual:**

1. `config/dual_camera_config.json` contiene una matriz genérica predeterminada: $$\mathbf{t} = [0.25, 0.0, 0.0]^T \quad (\text{separación fija de 25 cm sin rotación})$$
2. En la instalación física real del usuario:
3. La distancia entre cámaras calculada en hardware real es de aproximadamente **10.4 cm** ($\Delta t_x \approx -14.6\text{ cm}$ de error).
4. Existe una inclinación angular de convergencia ($\text{Yaw} / \text{Pitch}$) de aproximadamente **$20^\circ - 25^\circ$**.
5. Al aplicar una traslación errónea de 25 cm con $0^\circ$ de inclinación, la proyección de la Cámara 2 flota 15 cm a la derecha de la Cámara 1, provocando la duplicación completa del sujeto en lugar de superponerse.

---

## 2. Referencias de la Industria e Implementaciones Preexistentes

Para diseñar una solución sólida y comprobada, se analizaron tres implementaciones consolidadas en robótica, visión artificial y captura volumétrica:

### A. Referencia Oficial de Intel: `box_dimensioner_multicam`

- **Fuente:** Repositorio oficial de Intel `librealsense/wrappers/python/examples/box_dimensioner_multicam`.
- **Patrones Clave Adoptados:**
- `realsense_device_manager.py`: Configuración multihilo de dispositivos por número de serie y mitigación óptica del emisor infrarrojo (Master con emisor ON, Slave con emisor OFF).
- `calibration_kabsch.py`: Estimación de matriz de transformación rígida $T_{2 \to 1} = [R \mid \mathbf{t}]$ mediante correspondencia tridimensional y descomposición SVD (algoritmo de Kabsch).
- `measurement_task.py`: Proyección de la nube de la Cámara 2 en el espacio de la Cámara 1.

### B. Referencia de Captura Volumétrica en Tiempo Real: `LiveScan3D`

- **Fuente:** Proyecto de código abierto `MarekKowalski/LiveScan3D`.
- **Patrones Clave Adoptados:**
- *Filtro de Vóxeles en Espacio Compartido:* En lugar de sobreponer puntos, se discretiza el volumen común y se fusionan los puntos redundantes en un único vértice ponderado.
- *Visualización de Solape por Color de Sensor:* Mapeo diferencial de color por cámara (Cian para sensor 1, Magenta para sensor 2) para evaluar visualmente el ajuste de alineación antes de escanear.
- *Recorte de Volumen Bounding Box:* Eliminación de geometría fuera del rango de interés para maximizar la tasa de cuadros por segundo.

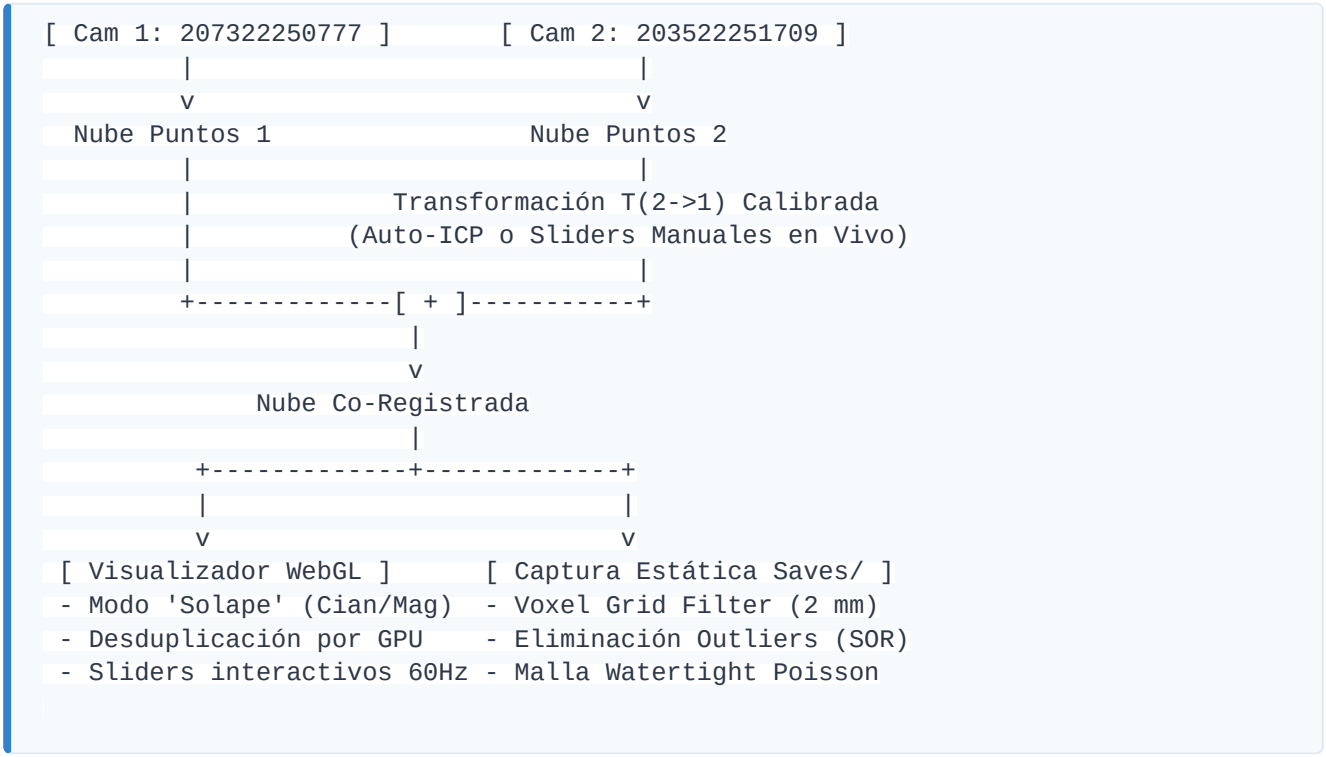
### C. Referencia de Fusión de Superficie Continua: Open3D `ScalableTSDFVolume`

- **Fuente:** Pipeline oficial Open3D RGB-D Integration (basado en Curless & Levoy 1996 y KinectFusion 2011).
- **Patrones Clave Adoptados:**
- Integración de funciones de distancia con signo truncada (*TSDF*) sobre vóxeles cúbicos de $2.5\text{ mm}$ a $3\text{ mm}$.
- Promediado continuo de distancia y color fotométrico en la zona de solape de múltiples cámaras.
- Extracción de una **única malla triangular hermética (watertight)** mediante *Marching Cubes*, resolviendo de raíz el problema de geometrías duplicadas o dobles paredes.

---

## 3. Arquitectura de la Solución

El objetivo es convertir dos señales de video 3D independientes en **un solo modelo geométrico continuo**, complementario y sin costuras, tanto en tiempo real (WebGL/Three.js) como en los escaneos exportados (Open3D/Blender).




---

## 3. Fases de Implementación en Código

### Fase 1: Motor de Calibración Automática sin Tablero (ICP Multiescala)

Permite al usuario calibrar las cámaras con un solo clic, sin requerir tableros impresos, calculando la pose relativa óptima a partir de la geometría común presente en la escena.

#### 1.1 Módulo Backend de Registro ( `live3d/registration.py` )

Implementar algoritmo de registro robusto multiescala en Open3D: 1. **Filtrado de Profundidad:** Limita los puntos a la zona de interés activa ($Z \in [0.35, 2.5]\text{ m}$). 2. **Normal Estimation:** Estimación de normales locales con búsqueda por radio ($r=0.03\text{ m}$). 3. **Registro en Cascada (Coarse-to-Fine):** * *Nivel 1 (Voxel 5 cm):* Búsqueda gruesa con traslación inicial aproximada. * *Nivel 2 (Voxel 2 cm):* Point-to-Plane ICP para corregir orientación y rotación angular. * *Nivel 3 (Voxel 5 mm):* Colored ICP (Park et al.) que combina geometría de profundidad y gradientes de color RGB para alineación sub-milimétrica. 4. **Validación de Calidad:** * Condición de éxito: $\text{Fitness} \ge 0.50$ y $\text{RMSE} \le 0.015\text{ m}$ (15 mm). * Retorna matriz $4 \times 4$ refinada y métricas de error.

#### 1.2 Endpoint API ( `live3d/server.py` )

- **Ruta:** `POST /api/dual/auto_align`
- **Flujo:**

- Captura snapshot sincronizado de ambas cámaras vía `src.get_snapshot()`.
- Ejecuta el registro en hilo secundario (`run_in_executor`).
- Si converge, actualiza las extrínsecas en memoria (`src.set_extrinsics(...)`) y las guarda en `config/dual_camera_config.json`.
- Devuelve `{ "status": "success", "matrix_4x4": [...], "fitness": 0.88, "rmse_mm": 4.2 }`.

---

### Fase 2: Panel de Calibración Manual e Interactiva en Tiempo Real (Frontend)

Para los casos en los que la escena tenga poca textura o el usuario quiera un ajuste visual milimétrico inmediato directamente en pantalla.

#### 2.1 Componentes UI (`live3d/web/index.html`)

Añadir subpanel colapsable `#dual-calibration-tools` bajo `#dual-cam-controls`: * **Botón Acción Principal:** `■ Auto-Alinear (ICP)` con indicador de carga y feedback de error/éxito. * **Sliders de Traslación Relativa:** * $T_x$ (Línea base horizontal): rango $[-40, 40]\text{ cm}$, paso $0.2\text{ cm}$. * $T_y$ (Altura vertical): rango $[-25, 25]\text{ cm}$, paso $0.2\text{ cm}$. * $T_z$ (Profundidad relativa): rango $[-30, 30]\text{ cm}$, paso $0.2\text{ cm}$. * **Sliders de Rotación Angular:** * $\text{Yaw}$ (Convergencia / ángulo horizontal): rango $[-45^\circ, 45^\circ]$, paso $0.5^\circ$. * $\text{Pitch}$ (Inclinación vertical): rango $[-30^\circ, 30^\circ]$, paso $0.5^\circ$. * $\text{Roll}$ (Rotación axial): rango $[-20^\circ, 20^\circ]$, paso $0.5^\circ$. * **Botón de Guardado:** `■ Guardar Calibración` (persiste la matriz afinada en el servidor). * **Botón de Restablecimiento:** `■ Restaurar`.

#### 2.2 Sincronización en Tiempo Real (`live3d/web/js/main.js` & `viewer.js`)

- Descomposición de matriz $4 \times 4 \leftrightarrow$ Vector traslación $[t_x, t_y, t_z]$ y ángulos Euler $[\psi, \theta, \phi]$.
- Evento `input` en cada slider:
- Reconstruye la matriz $4 \times 4$ en JavaScript instantáneamente.
- Llama a `viewer.setExtrinsics(m4x4)`.
- La Cámara 2 se mueve a 60 FPS en el canvas Three.js sin latencia ni llamadas de red intermedias.
- Al abrir el panel de calibración, se conmuta automáticamente al modo de color **"Solape"**:
- Cam 1 = Cian brillante.
- Cam 2 = Magenta brillante.
- El usuario mueve los sliders hasta que el cian y el magenta coincidan en su rostro y torso, formando una sola silueta coherente.
- El botón `Guardar Calibración` envía `POST /api/dual/extrinsics` con la matriz resultante.

---

### Fase 3: Pipeline de Fusión y Desduplicación en Captura (`capture_dual_scan`)

Garantizar que los escaneos estáticos guardados en Saves/ contengan **un solo resultado unificado sin puntos duplicados**.

#### 3.1 Proceso de Fusión en `live3d/capture.py`:

1. **Transformación Rígida:** $$P_2^{\text{world}} = T_{2 \to 1} \cdot P_2$$
2. **Refinamiento ICP Fino (Opcional por cuadro):** Ajuste residual entre $P_1$ y $P_2^{\text{world}}$ con umbral de 3 cm para compensar pequeñas vibraciones del soporte.
3. **Filtro de Vóxeles Unificado (Voxel Grid Downsampling):** `python pcd_unificada = (pcd1 + pcd2_transformed).voxel_down_sample(voxel_size=0.0025)`
4. *Mecanismo:* Divide el espacio en cubos de $2.5\text{ mm}$.
5. En la zona de solape, los puntos duplicados de ambas cámaras que caen en el mismo cubo se condensan en **un único punto centroidal**: $$\mathbf{p}_{\text{fused}} = \frac{1}{N} \sum^N \mathbf{p_i, \quad \mathbf{c}_i$$} = \frac{1}{N} \sum_{i=1}^N \mathbf{c
6. Se elimina al 100% la redundancia de puntos y la sobre-densidad en el área compartida.
7. **Filtrado Estadístico de Outliers (SOR):** Elimina ruido flotante y puntos dispersos en los límites de campo de visión: `python pcd_limpia, _ = pcd_unificada.remove_statistical_outlier(nb_neighbors=24, std_ratio=1.2)`
8. **Reconstrucción de Malla Continua:**
9. Cálculo de normales consistentes con orientación hacia la cámara correspondiente.
10. Generación de malla superficial mediante *Poisson Surface Reconstruction* (`depth=9`) o *Ball Pivoting*.
11. Guardado de `malla_fusionada.obj` y `nube_fusionada.ply`.
12. **Exportación de Diagnóstico:**
13. Guardado de `diagnostico_solape.ply`: Nube donde los puntos de Cam 1 se tiñen de cian puro y los de Cam 2 de magenta puro, permitiendo verificar la perfección de la unión en Blender o MeshLab.

---

## 4. Estructura de Cambios y Archivos Afectados

| Archivo | Rol | Modificaciones |
|---|---|---|
| `live3d/registration.py` *(Nuevo)* | Motor ICP | Implementación de `register_dual_rgbd` (Multi-scale Point-to-Plane & Colored ICP). |

| Archivo | Rol | Modificaciones |
| --- | --- | --- |
| `live3d/server.py` | API REST | Endpoint `POST /api/dual/auto_align` y actualización de extrínsecas. |
| `live3d/capture.py` | Pipeline Fusión | Fusión con Voxel Downsampling, refinamiento ICP y eliminación de duplicados en `capture_dual_scan`. |
| `live3d/web/index.html` | UI Frontend | Sliders de traslación ($X, Y, Z$), rotación (Yaw, Pitch, Roll) y botón `Auto-Alinear (ICP)`. |
| `live3d/web/js/main.js` | Lógica Cliente | Enlace bidireccional entre sliders, matriz $4 \times 4$, Three.js y persistencia en backend. |
| `live3d/web/js/viewer.js` | Render Three.js | Helper para descomponer matriz a Euler y aplicar transformaciones instantáneas. |

---

## 5. Criterios de Aceptación y Validación

1. **Visor en Vivo:**
2. En modo "Solape", las dos siluetas (Cian y Magenta) se alinean perfectamente en un solo cuerpo sin duplicación.
3. El ajuste de sliders actualiza la posición en Three.js con fluidez ($> 30\text{ FPS}$).
4. **Auto-Alineación ICP:**
5. Al pulsar `Auto-Alinear (ICP)`, el sistema calcula la pose en $< 1.5$ segundos y alinea automáticamente ambas cámaras.
6. **Escaneo Guardado en Saves/:**
7. El archivo `nube_fusionada.ply` contiene un único sujeto con densidad homogénea en la zona de solape.
8. Inspección visual en Blender confirma ausencia de superficies dobles o dobles rostros.
9. **Persistencia:**

10. La calibración guardada persiste entre reinicios del servidor en `config/dual_camera_config.json`.